

Kurs Python for AI

Sprawdzenie wersji Python w terminalu

```
python3 --version
```

Visual Studio Code (VS Code)

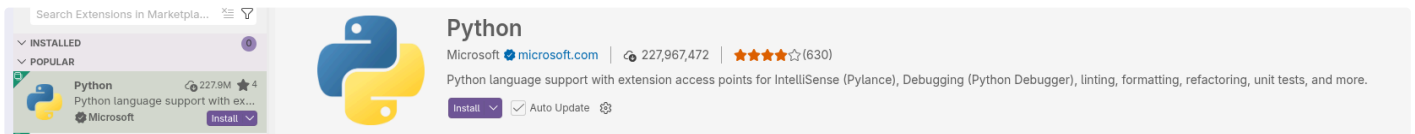
- na bazie VS Code zbudowano Cursor;
 - Cursor to AI-powered coding assistant;
 - Microsoft is the owner and developer;
 - core source code is open-source under the MIT License and hosted on GitHub
-

Why I need to add VS Code app to the PATH?

- PATH is an environment variable;
 - Allows you to run its executable commands from any directory without typing the full file path;
-

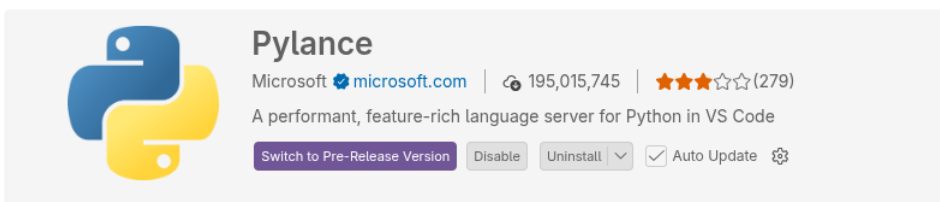
VS Code Extensions

Python



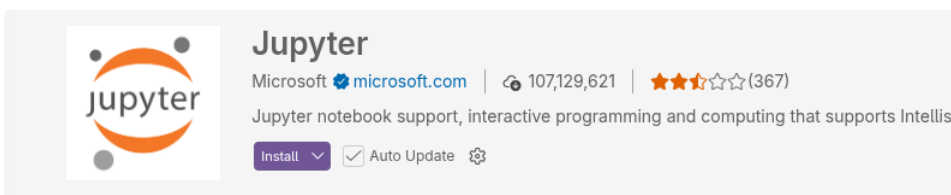
- developed by Microsoft;
 - transforms VS Code into a comprehensive Python development environment;
 - bridge between the VS Code editor and the local Python interpreter;
 - enabling code execution, testing, and environment management;
 - installing the extension automatically triggers the installation of Python Debugger and Pylance
-

Pylance



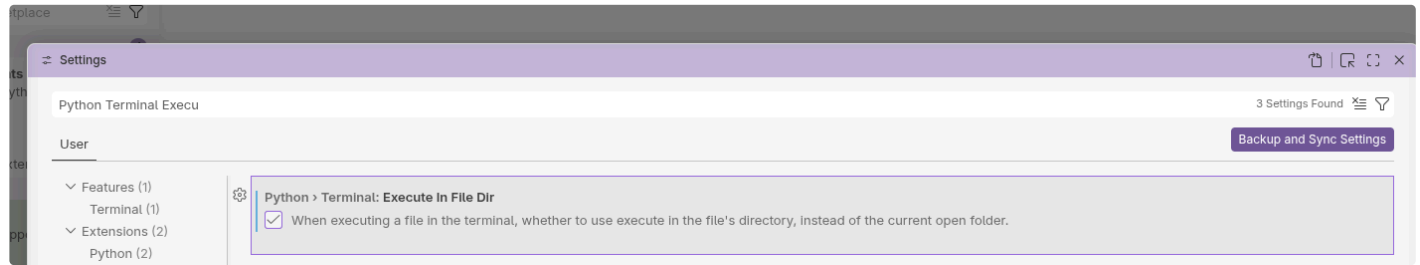
- high-performance language server extension;
 - provides advanced Python language support;
 - runs as a background process that analyzes your code to provide real-time feedback and assistance;
 - code completion and error detection;
 - works alongside the Python extension;
-

Jupyter



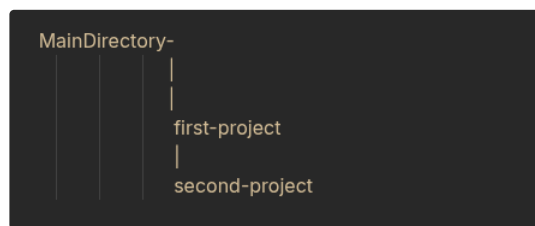
Configure Python execution in VSCode

- in Settings type in: *Python Terminal Execute In File Dir*



- VS Code will execute Python script from the file's directory instead of workspace root;
- prevents common path-related errors;
- if your script reads a file with `open('data.csv')`, it will look for the file in the same folder as your script, which is usually what you want. Without this setting, it would look in your project root instead, causing "file not found" errors;

Struktura Projektu



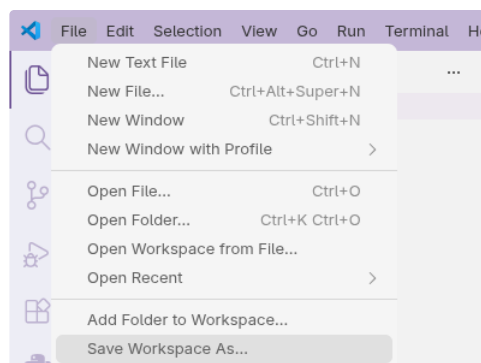
- przykład: PythonProjects ---> python-for-ai
- use lowercase letters with dashes (called "kebab-case") like `python-for-ai` or `my-first-project`. This matches how projects appear on GitHub.

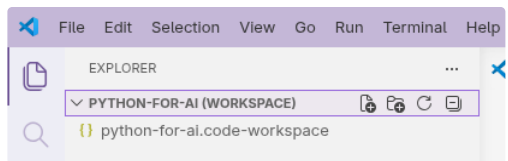
```
lukasz@fedora:~/PythonProjects$ tree
.
├── python-for-ai
└── 
```

2 directories, 0 files
lukasz@fedora:~/PythonProjects\$

Stwórz Workspace

- po otwarciu folderu, w którym zamierzam przechowywać projekt należy zapisać ten folder jako workspace;
- workspace is like a bookmark for your project;
- workspace saves your project settings and makes it easy to return to your project later;
- it remembers:
 - which folder you're working in;
 - your editor settings for this project;
 - which files you had open;
 - your debugging configuration.





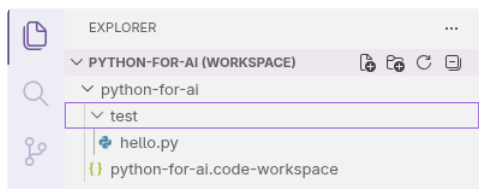
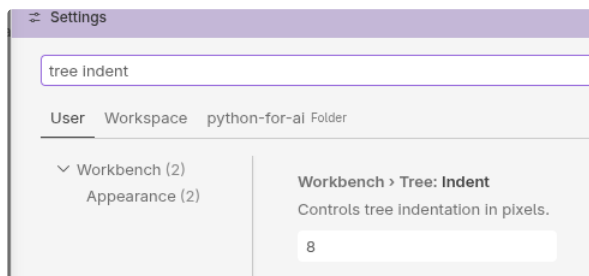
Home / PythonProjects / python-for-ai



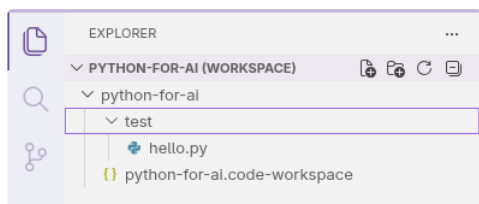
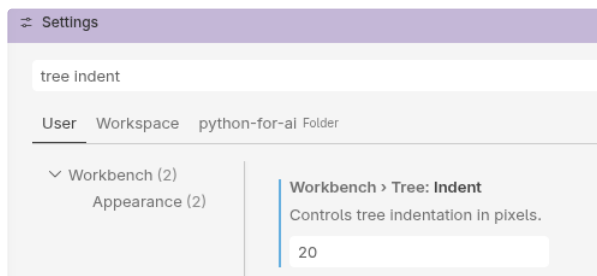
- po ustawieniu "workspace" mogę zamknąć VS Code, następnie przejść do katalogu w którym zapisałem workspace i zrobić double-click na skrót - VS Code otworzy się samo w miejscu, w którym ostatnio pracowałem

Ustaw tree indent

- zwiększa czytelność struktury drzewa katalogów



- zwiększenie do 20 pixeli:



Uruchamianie kodu z terminala

```
lukasz@fedora:~/PythonProjects/python-for-ai$ python hello.py
Hello World!
lukasz@fedora:~/PythonProjects/python-for-ai$
```

Virtual Environment

- wyizolowane środowisko wirtualne dla konkretnego projektu

Po co to jest?

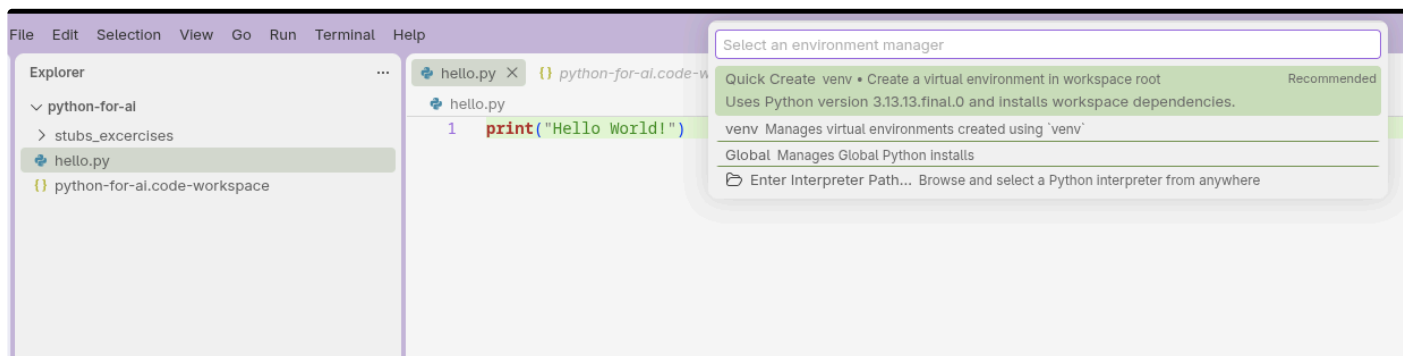
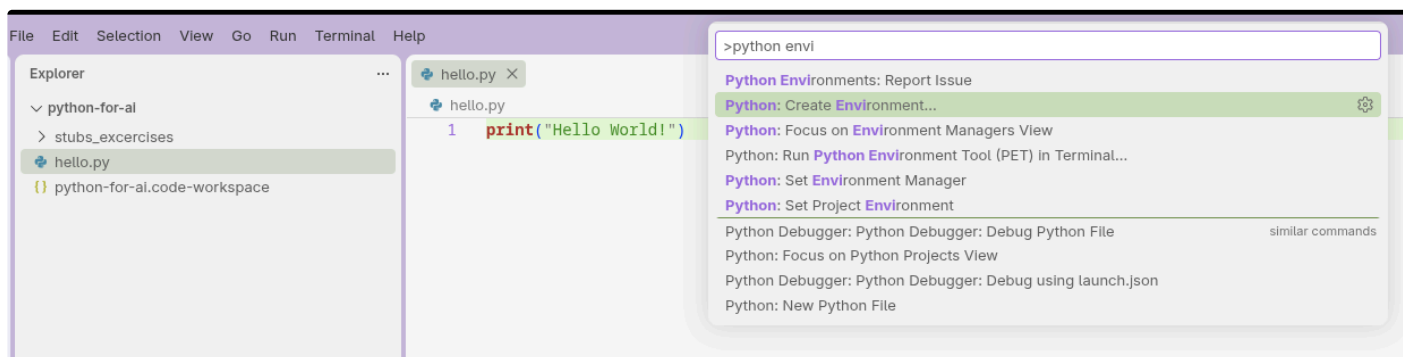
Wyobraź sobie, że pracujesz nad dwoma projektami:

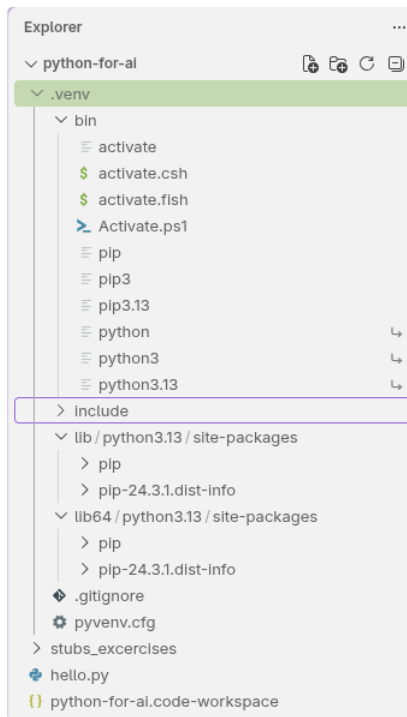
- Projekt A potrzebuje `django==3.2`
- Projekt B potrzebuje `django==4.2`

Jeśli instalujesz pakiety globalnie (na cały system), masz problem — nie możesz mieć dwóch wersji tego samego pakietu naraz. **venv** rozwiązuje to, tworząc osobny, odizolowany "folder" z własnym interpreterem Pythona i własnymi pakietami dla każdego projektu.

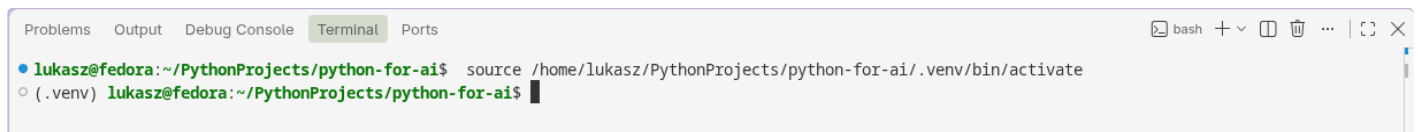
Tworzenie virtual environment z poziomu VSCode

- CTRL+SHIFT+P --- otwiera "command palette"





- Po uruchomieniu nowego terminala venv powinien zostać aktywowany przez VSCode



Tworzenie venv z poziomu terminala

```
python -m venv nazwa_srodowiska
```

Najczęściej nazywa się je po prostu `venv` lub `.venv`.

2. Aktywacja:

Linux/Mac:

```
source venv/bin/activate
```

Windows:

```
venv\Scripts\activate
```

3. Instalacja pakietów (tylko wewnątrz venv):

```
pip install requests pandas
```

4. Dezaktywacja:

```
deactivate
```

```
lukasz@fedora:~/PythonProjects/python-for-ai$ ls
hello.py  python-for-ai.code-workspace  stubs_exercises
lukasz@fedora:~/PythonProjects/python-for-ai$ python -m venv venv
lukasz@fedora:~/PythonProjects/python-for-ai$ ls
hello.py  python-for-ai.code-workspace  stubs_exercises  venv
lukasz@fedora:~/PythonProjects/python-for-ai$ source venv/bin/activate
(venv) lukasz@fedora:~/PythonProjects/python-for-ai$
(venv) lukasz@fedora:~/PythonProjects/python-for-ai$ pwd
/home/lukasz/PythonProjects/python-for-ai
(venv) lukasz@fedora:~/PythonProjects/python-for-ai$ deactivate
lukasz@fedora:~/PythonProjects/python-for-ai$
lukasz@fedora:~/PythonProjects/python-for-ai$ source venv/bin/activate
(venv) lukasz@fedora:~/PythonProjects/python-for-ai$
```

Dlaczego warto używać virtual environment ?

- **Izolacja** — pakiety jednego projektu nie kolidują z innymi
- **Czystość** — nie zaśmiecasz globalnej instalacji Pythona
- **Reprodukowalność** — możesz łatwo odtworzyć środowisko na innym komputerze (np. przez `requirements.txt`)
- **Bezpieczeństwo** — nie musisz mieć uprawnień administratora, żeby instalować pakiety